01

MGU BCA - 3RD SEM

Software Engineering

Module 1 - Process and Agile

By Aben Sojan

Exam - 10 Nov 2026

Complete question bank: every important, repeated and sure-mark question for this module, each with a full-mark model answer. Nothing important left out.

EXAM PATTERN (from real MGU question papers)

SF HONOURS EXAM PATTERN - 50 marks, 1.5 hours. Part A: 10 x 2 marks, all compulsory; Q1-Q4 come from this module (process and agile). Part B: 5 x 5 marks, answer any 3 of 5. Part C: 2 x 15 marks, answer any 1 of 2 (model paper essays: requirements elicitation, system testing). Old papers (May 2019 - Feb 2025) repeat SDLC questions verbatim: life-cycle activities 5x, waterfall 3x, Iterative-vs-RAD identical in Mar 2021 and Aug 2022. Attempt at least one question from each course outcome (CO1 = this module).

TOPIC 1

Evolving Role and Changing Nature of Software

1

Software started as simple programs written by one person; now it runs banks, hospitals, phones and the whole internet.

Software moved from custom-built programs to mass products, then to web apps, mobile apps and AI systems.

Today software is the most important part of almost every business.

QUESTION BANK

1 Explain the evolving role of software.

IMPORTANT

Software is the set of programs and data that tell a computer what to do, and its role in society has changed completely over time.

In the early days, software was small and written by one person for one task, like a simple calculation program used by its own writer.

Then software became a product sold to many users, like operating systems and office tools.

Later, software moved to the internet as web applications that millions of people could use from anywhere.

Then came mobile apps, which put software in the pocket of almost every person on earth.

Today software runs banks, hospitals, transport, phones and the whole internet.

Cloud systems let companies use powerful software without owning any hardware.

Artificial intelligence systems now make decisions and create content like a human assistant.

So software changed from a small tool into the most important part of almost every business.

Example: a bank today is really a software company, because every account and payment is handled by software.

In short, software has grown from a helper program into the backbone of the modern world.

2 Explain the changing nature of software.

IMPORTANT

The nature of software means how it is built, who uses it and how it behaves, and this has changed a lot over time.

Old software was built for a single user and ran on one machine.

Modern software serves millions of users at the same time over the internet.

Old software was updated rarely, maybe once a year with a new version.

Modern software is updated continuously, sometimes many times a day.

Old software was a support tool that helped people do other work.

Today software is often the main product of the company itself, like a food delivery app or a video streaming service.

Modern software connects to other software through the internet and shares data constantly.

It must handle security, speed and huge amounts of data, which old software never faced.

Example: a simple calculator app of the past only added numbers, but a modern app like Google Pay moves real money safely for crores of users.

So the nature of software has changed from simple standalone tools to large, connected, always-changing systems.

3 Explain the importance of software engineering.

IMPORTANT

Software engineering is important because modern software is too large and complex to build by trial and error.

It gives a disciplined, step-by-step method for delivering software on time and within budget.

Without it, projects suffer missed deadlines, crossed budgets and poor quality.

It applies engineering ideas like planning, measurement and quality checks to software work.

It helps teams handle changing requirements without falling into chaos.

It reduces cost because finding and fixing an error early is far cheaper than fixing it after delivery.

It improves quality through reviews, testing and clearly defined processes.

It makes maintenance easier, since well-documented software can be changed by people who did not write it.

Example: a hospital management system built with proper requirements and testing will not lose patient records, while one built casually might.

In short, software engineering turns software building from risky guesswork into a controlled,

repeatable activity.

4 Describe the essential attributes of good software.

IMPORTANT

Good software has a set of essential attributes that make it useful and trustworthy.

Maintainability: it should be easy to change and fix when needs change.

Reliability: it should work correctly without frequent failures.

Efficiency: it should use time and memory wisely instead of wasting resources.

Usability: it should be easy to learn and pleasant to use.

Security: it should protect data from misuse and attacks.

Portability: it should run on different machines with little change.

These attributes often trade off against each other, so the team must balance them for the situation.

Example: a banking app is good software only if it is reliable with money, secure against fraud, and simple enough for anyone to use.

In short, good software is not just working code; it is maintainable, reliable, efficient, usable and secure.

5 Distinguish between generic software products and customized products.

IMPORTANT

Generic products are built for a wide market of customers; customized products are built for one specific customer.

For generic products, the requirements are decided by the developers through market study, as with Microsoft Word.

For customized products, the requirements are given by the customer under a contract, as with a billing system for one hospital.

Generic products are sold to many buyers; customized products are delivered to a single buyer.

Testing a generic product must cover many kinds of users and environments.

Testing a customized product focuses on the one client's workflows.

Generic products earn through volume sales; customized products earn through the development contract and maintenance.

Example: Tally is a generic accounting product, while the railway reservation system is a customized product built for Indian Railways.

In short, generic means made for the market, customized means made for one customer.

TOPIC 2

2

Layered Technology

Software engineering is built in layers, like floors of a building.

Bottom layer: tools (editors, compilers, testing tools).

Middle layer: methods (how to plan, design, test).

Top layer: process (the overall framework that holds everything).

Quality focus sits at the base and supports all layers.

QUESTION BANK

6 **Explain layered technology in software engineering.**

IMPORTANT

Software engineering is built in layers, like the floors of a building, where each layer rests on the one below it.

The top layer is the process layer, which defines the overall framework of activities used to build software.

The middle layer is the methods layer, which gives the technical how-to for each activity, like how to design or test.

The bottom layer is the tools layer, which gives automated support for the work, like editors, compilers and testing tools.

Beneath all three layers sits the quality focus, which supports everything and makes sure quality is checked at every step.

The layers depend on each other: a good process is useless without proper methods, and methods are slow without tools.

Example: for a project, the process could be Scrum, the method could be use-case modeling for design, and the tool could be Git for version control.

If any layer is weak, the whole building shakes, so all layers must be strong together.

In short, layered technology organises software engineering into process, methods and tools, all standing on quality.

7 **What are the three layers of software engineering technology?**

IMPORTANT

The three layers are process, methods and tools, supported by a quality focus at the base.

The process layer is the top layer; it defines the framework of activities, like communication, planning, modeling, construction and deployment.

It organises all the work in the correct order from start to finish.

The methods layer is the middle layer; it gives the technical how-to for each activity, like how to do analysis, design, coding or testing.

Example: writing UML diagrams is a design method.

The tools layer is the bottom working layer; it gives automated support, like IDEs, compilers, debuggers and test runners.

Example: Visual Studio Code and Git are tools.

The quality focus is not a fourth layer but the foundation that supports all three, keeping quality checks active everywhere.

Together, process tells what to do and when, methods tell how to do it, and tools help do it faster and with fewer errors.

TOPIC 3

Process Framework

3

A process framework is the skeleton every software project follows.

It has framework activities: communication, planning, modeling, construction and deployment.

Each activity has actions and tasks; umbrella activities like quality assurance run through all of them.

Different process models adapt this framework in different ways.

QUESTION BANK

8 **What are the life-cycle phases? Explain the life-cycle activities of software development. [Asked: May 2019, Mar 2021, Oct 2021, Aug 2022, Feb 2025]**

SURE MARK

The software life cycle is the series of phases a product passes through from birth to retirement.

The main life-cycle phases are requirements analysis, design, coding, testing, deployment and maintenance.

Requirements analysis discovers what the customer actually needs.

Design plans how the software will be built: its structure, modules and screens.

Coding is the actual writing of the program from the design.

Testing checks that the software works correctly and meets the requirements.

Deployment installs the software for real users.

Maintenance fixes problems and adds new features after delivery.

Each phase ends with a review, so errors are caught before they flow into the next phase.

Example: for a railway ticket app, the team first studies booking needs, then designs screens, codes the app, tests booking flows, releases it, and later adds seat-map features.

In short, the life-cycle phases carry software from an idea to a working product and keep it alive afterwards.

9 **Define software process. [Asked: Aug 2022, Feb 2025]**

REPEATED

A software process is the set of related activities that leads to the production of a software product.

It defines which activities are done, in what order, and who performs them.

Typical activities include specification, design and implementation, validation, and evolution.

Every software project follows some process, whether it is written down or followed informally.

A defined process makes the work predictable, repeatable and controllable.

Without a process, projects depend on individual heroics and frequently miss deadlines.

Different projects need different processes: waterfall suits stable requirements, agile suits changing ones.

The choice depends on project size, team skill, risk and customer involvement.

Example: a college building an admission portal with fixed requirements can safely use the waterfall process.

In short, the software process is the roadmap a team follows from an idea to working software.

10 What is a process framework? Explain its activities.

IMPORTANT

A process framework is the common skeleton of activities that every software project follows, whatever model it uses.

It defines five framework activities.

Communication: talk to the customer and stakeholders to understand what they need.

Planning: decide the work to do, the schedule, the cost and the risks.

Modeling: create models of what to build and how it will work, like analysis and design models.

Construction: write the code and test it to find and fix errors.

Deployment: deliver the software to the customer and collect feedback for improvement.

Alongside these, umbrella activities run through the whole project.

Umbrella activities include project tracking, risk management, quality assurance and configuration management.

Example: quality assurance is not a separate stage; it checks the work done in every activity, from planning to deployment.

Different process models like waterfall and agile adapt this same framework in their own way.

11 Distinguish between framework activities, actions and tasks.

IMPORTANT

Framework activities, actions and tasks are three levels of work in a process framework, from big to small.

Framework activities are the main stages of a project, like communication, planning, modeling, construction and deployment.

They are the same for every project and give the big picture.

Actions are smaller steps inside one framework activity.

Example: inside the planning activity, actions include creating a project schedule and doing risk analysis.

Tasks are the smallest units of work, usually done by one person in a short time.

Example: writing the login screen test cases is a single task inside the testing action.

So one framework activity contains many actions, and one action contains many tasks.

This division helps managers plan and track work at the right level of detail.

12 **Differentiate product and process.**

IMPORTANT

Product is what is built; process is how it is built.

The product is the final software delivered to the customer, like an app or a website.

The process is the set of activities used to create that product, like design, coding and testing.

Product quality is judged by the user; process quality is judged by how smoothly the work flows.

A good process usually leads to a good product, but a good product can sometimes emerge from a weak process by luck.

Products are measured by features, speed and reliability; processes are measured by cost, time and repeatability.

Organisations improve the process so that every future product comes out better.

Example: the banking app on your phone is the product; the Scrum sprints the team used to build it are the process.

In short, the product is the destination and the process is the road taken to reach it.

TOPIC 4

Waterfall Model

4

Waterfall is the oldest model: work flows down in fixed stages, one after another.

Stages: requirements, design, implementation, testing, deployment, maintenance.

You finish one stage fully before the next starts; going back is difficult and costly.

Best when requirements are clear and will not change.

QUESTION BANK

13 **What are the drawbacks of the waterfall model? [Asked: Mar 2021, Feb 2025]**

REPEATED

The biggest drawback of the waterfall model is that real projects rarely have fixed requirements at the start.

Customers often change their minds, and in waterfall any change after a stage is done is very expensive.

The customer sees working software only at the very end of the project.

If the final product is wrong, months of work may be wasted.

There is no early feedback, so misunderstandings are found very late.

The model does not handle risks well during development, because testing and integration

happen only near the end.

It also produces a lot of documents, which slows down small and fast-moving projects.

Example: if a shopping app is built in waterfall and the customer wanted a different payment option, the team must redo design and code from scratch.

So waterfall is risky whenever requirements are unclear or likely to change.

14 How do you select an appropriate life-cycle model for a project? Illustrate with examples. [Asked: May 2019, Feb 2025]

REPEATED

The life-cycle model is selected by matching the project's nature with each model's strengths.

First, check requirement stability: stable requirements suit waterfall; changing ones suit agile or spiral.

Second, check project size and risk: large, risky projects suit the spiral model, which handles risk in every loop.

Third, check customer involvement: if the customer can review often, prototyping or agile works well.

Fourth, check deadlines: tight deadlines with clear requirements suit RAD or incremental delivery.

Fifth, check team skill: agile needs experienced, self-managing teams.

Sixth, check how critical failure is: life-critical systems need the heavy verification of plan-driven models.

Example: a fixed-requirement exam-result portal suits waterfall; a startup's experimental app suits Scrum; a satellite control system suits spiral because failure is not an option.

In short, there is no best model for all projects; the right model is the one that fits the project's requirements, risks and people.

15 Explain the waterfall model with its stages.

IMPORTANT

The waterfall model is the oldest and simplest process model, where work flows down through fixed stages like water falling over steps.

The first stage is requirements: gather and document everything the customer needs.

The second stage is design: plan the software structure based on the requirements.

The third stage is implementation: write the actual code.

The fourth stage is testing: check the software for errors and fix them.

The fifth stage is deployment: deliver the finished software to the customer.

The sixth stage is maintenance: fix problems and make small improvements after delivery.

Each stage must be fully finished before the next one starts.

Going back to an earlier stage is difficult and costly, so requirements must be clear from the beginning.

Example: building software for an ATM machine works well, because the rules for cash withdrawal are fixed and well known.

In short, waterfall is simple and easy to manage, but only fits projects where requirements will not change.

16 Compare waterfall model with agile development.

IMPORTANT

Waterfall and agile are opposite styles of building software.

Waterfall moves in fixed stages in one direction: requirements, design, coding, testing and deployment.

Agile works in small cycles called iterations or sprints, repeating planning, building and testing many times.

In waterfall, change is costly because a finished stage must be redone.

In agile, change is welcome even late in the project, because only small pieces are rebuilt.

In waterfall the customer sees working software only at the end.

In agile the customer gets working software every few weeks and can give feedback early.

Waterfall needs heavy documentation; agile keeps documentation light and focuses on working software.

Example: a government tax system with fixed rules suits waterfall, while a new food delivery app with changing ideas suits agile.

In short, waterfall fits stable requirements and agile fits changing requirements.

17 When is the waterfall model suitable?

IMPORTANT

The waterfall model is suitable when the requirements are well understood and unlikely to change.

It works best for small projects with a fixed scope and clear goals.

It suits projects that need strict documentation, like banking or medical systems.

It is good when the technology is familiar and the team knows exactly how to build it.

It is also useful when the customer cannot give regular feedback, since the model does not depend on it.

In such cases the simplicity of waterfall becomes a strength, because the plan can be followed step by step.

Example: software for a simple billing system with fixed tax rules is a perfect fit.

The opposite case is a startup building a new social app, which should avoid waterfall because its ideas will change often.

So waterfall is suitable when the road is clear, fixed and well mapped in advance.

18 Write a note on project risks in the selection of a life-cycle model.

IMPORTANT

Choosing the wrong life-cycle model is itself a major project risk.

If requirements are unstable and waterfall is chosen, late changes become extremely expensive.

If the team is inexperienced and agile is chosen without guidance, sprints can drift without direction.

Technical risks, like unproven technology, need a model with early prototypes or risk loops, such as the spiral model.

Schedule risks need models that deliver early usable versions, like incremental or RAD.

Customer-related risks, like an unavailable customer, rule out models needing constant feedback, such as XP.

The selection should therefore start with a risk assessment, not with habit or fashion.

Example: picking waterfall for a hospital system whose rules keep changing risks delivering a system nobody wants.

In short, model selection must weigh requirement, technical, schedule and people risks before the project starts.

TOPIC 5

5

Incremental Model

Incremental model builds software in small pieces called increments.

The first increment is a basic working version; later increments add more features.

Each increment goes through requirements, design, coding and testing.

The customer gets usable software early and gives feedback.

QUESTION BANK

19 **What are the advantages of the incremental model? [Asked: May 2019, Feb 2025]**

REPEATED

The first advantage of the incremental model is early delivery: the customer gets working software quickly instead of waiting for the whole project.

Early delivery means the customer can start getting value and testing the product sooner.

The second advantage is feedback: comments on early increments improve the later ones.

The third advantage is lower risk, because problems are found in small pieces instead of one big failure at the end.

It is also easier to handle changes, since new requirements can be added to a future increment.

Testing is simpler because each small increment is tested alone before joining the system.

The team and customer both stay motivated, because progress is visible after every increment.

Example: in a railway booking site, the ticket booking increment can go live while the refund increment is still being built.

So the incremental model gives speed, flexibility and safety together.

20 **Explain the incremental process model.**

IMPORTANT

The incremental model builds software in small pieces called increments instead of all at once.

The first increment is a basic working version with only the core features.

Each later increment adds more features until the full system is complete.

Every increment goes through the same mini-process: requirements, design, coding and testing.

The customer gets usable software very early and can start using it.

Feedback on early increments is used to improve the later ones.

If one increment has a problem, it does not break the whole project, because the work is in

small pieces.

Example: a college portal can first release login and notices, then add fee payment in the second increment, and results in the third.

In short, incremental delivery gives early value, early feedback and lower risk.

TOPIC 6

6

Evolutionary Models: Prototyping and Spiral

Evolutionary models accept that software grows and changes over time.

Prototyping: build a quick rough version to understand unclear requirements, then build the real system.

Spiral model: work moves in loops; each loop plans, analyses risk, builds and reviews.

The spiral model is risk-driven: high-risk parts are handled first.

QUESTION BANK

21 **Compare the Iterative model with the RAD model. [Asked: Mar 2021, Aug 2022]**

REPEATED

Both build software in repeated cycles, but they differ in goal and method.

Iterative model: each iteration adds new features to a growing product; the focus is gradual refinement.

RAD: each cycle aims to deliver a complete working system fast, using reusable components.

The iterative model suits projects where requirements are not fully known at the start.

RAD suits projects where requirements are well understood and speed is the priority.

The iterative model manages risk by reviewing the growing product after every iteration.

RAD manages time by fixing short cycles and reusing proven parts.

The iterative model needs strong customer feedback between iterations.

RAD needs highly skilled developers and a library of reusable components.

Example: a new social media app fits the iterative model, while a standard company payroll system fits RAD.

In short, the iterative model grows the product step by step, while RAD races to a finished system using ready parts.

22 What are evolutionary process models? Explain any two.

IMPORTANT

Evolutionary models accept that software grows and changes over time instead of being fixed at the start.

They build the software in versions that improve with customer feedback.

The first example is prototyping.

In prototyping, a quick rough version of the software is built and shown to the customer.

This helps everyone understand unclear requirements before the real system is built.

The prototype is usually thrown away, and the final system is built after the requirements are clear.

The second example is the spiral model.

The spiral model works in repeated loops; each loop has planning, risk analysis, engineering and customer review.

It is risk-driven, so the most dangerous and uncertain parts are handled first.

Example: for a new online exam system, a prototype can show how the exam screen should look, and the spiral model can manage the risk of handling thousands of students at once.

Both models use feedback and repetition to grow the software safely.

23 What is prototyping? When is it used?

IMPORTANT

A prototype is a quick rough model of the software that is shown to the customer before the real system is built.

It looks and partly works like the final product, but it is not the final product.

Prototyping is used when the customer cannot explain clearly what they want.

It is used when requirements are confusing or new technology is involved.

It is also used when the user interface is important and must be approved early.

The customer tries the prototype, gives feedback, and the requirements become clear.

The prototype is usually thrown away after this, and the real system is built properly.

This saves money, because fixing a misunderstanding in a rough prototype is far cheaper than fixing finished code.

Example: for a hospital management app, a prototype can show the patient registration screen so doctors can say what they actually need.

In short, prototyping turns unclear ideas into clear requirements.

24 Explain the spiral model and its advantages.

IMPORTANT

The spiral model was proposed by Barry Boehm and works in repeated loops called spirals.

Each loop passes through four parts.

Planning: decide the goals and limits for this loop.

Risk analysis: find the biggest risks and decide how to reduce them; this is the heart of the spiral model.

Engineering: build and test a part of the software.

Customer review: show the result to the customer and plan the next loop.

The main advantage is risk handling: high-risk parts are faced first, so big failures are avoided.

Another advantage is early customer feedback in every loop.

It suits large, complex and high-risk projects very well.

The drawback is that it needs expert risk analysis, which makes it costly for small projects.

Example: software for a satellite launch would use the spiral model because the risk of failure is huge.

So the spiral model trades extra planning effort for safety on big risky projects.

25 What is RAD? Explain the RAD model.

IMPORTANT

RAD stands for Rapid Application Development, a model that builds software very fast using reusable components.

It works in short cycles of about 60 to 90 days, delivering a working system quickly.

RAD has four phases: business modeling, data modeling, process modeling, and application generation and testing.

Business modeling studies what information drives the business.

Data modeling organises that information into data objects.

Process modeling describes how the data is used to complete business tasks.

Application generation builds the system using ready-made components and automated tools, then tests it.

RAD needs a highly skilled team and works best when requirements are well understood.

Example: building a standard inventory system for a shop chain suits RAD because the requirements are clear and components can be reused.

In short, RAD trades detailed upfront planning for speed by reusing components in short, sharp cycles.

26 **If requirements are frequently changing, which life-cycle model should be selected? Justify.**

IMPORTANT

When requirements change frequently, an agile or evolutionary model should be selected.

The waterfall model fails here because it freezes requirements early, making later changes very costly.

Agile models like Scrum welcome changing requirements even late in development.

They work in short sprints, so new or changed requirements simply enter the next sprint backlog.

Evolutionary models like prototyping also fit, because the customer sees working versions early and corrects course.

The spiral model fits too, since each loop re-examines risks, including the risk of requirement change.

The key test is: does the model expect change or punish it?

Example: a startup building a food-delivery app with shifting ideas should use Scrum, adding new features sprint by sprint.

In short, pick a model that expects change: agile or evolutionary, never a rigid plan-driven one.

TOPIC 7

Unified Process

7

The Unified Process (UP) is an iterative model driven by use cases.

It has four phases: inception, elaboration, construction, transition.

Work is organised into workflows like requirements, analysis, design, implementation and test.

It is architecture-focused and widely used for object-oriented projects.

QUESTION BANK

27 Explain the Unified Process with its phases.

IMPORTANT

The Unified Process, or UP, is an iterative process model that is driven by use cases.

Use cases describe how users interact with the system, and they guide all the work.

UP is also architecture-focused, meaning the basic structure of the system is fixed early.

It has four phases.

Inception: understand the idea, define the scope, and get approval to continue.

Elaboration: design the architecture, remove the major risks, and make a detailed plan.

Construction: build the software in iterations until it is complete.

Transition: deliver the software to users, train them and fix the first problems.

Work is organised into workflows like requirements, analysis, design, implementation and test that run across the phases.

UP suits large object-oriented projects that need a strong and stable architecture.

Example: a bank building a new core banking system would use UP to keep the architecture solid from the start.

28 What happens in each phase of the Unified Process?

IMPORTANT

Each phase of the Unified Process has a clear goal and a clear result.

Inception: the team defines the project scope, lists the main use cases, checks if the project is possible, and gets approval.

Its result is a yes-or-no decision to continue.

Elaboration: the team designs the system architecture, builds a working skeleton, removes major technical risks and makes the full plan.

Its result is a stable architecture everyone agrees on.

Construction: the team builds the software in iterations, adding features and testing them each time.

Its result is a complete and tested system.

Transition: the team delivers the software to users, trains them, fixes early problems and hands over documentation.

Its result is a live system running with real users.

Example: in a railway reservation project, inception approves the idea, elaboration designs how booking works, construction builds it, and transition puts it in stations.

So the four phases move the project safely from idea to live use.

TOPIC 8

8

Agility Principles and Agile Values

Agility means the ability to respond quickly to change.

Agile principles: satisfy the customer with early delivery, welcome changing requirements, deliver working software frequently.

People and interactions matter more than processes and tools.

Working software matters more than heavy documentation.

QUESTION BANK

29 **Distinguish between plan-driven and agile development. [Asked: Model paper]**

REPEATED

Plan-driven and agile are two opposite ways of managing software projects.

In plan-driven development, the full plan is made first: requirements, design and schedule are fixed before coding.

In agile, planning adapts as work continues; only the near future is planned in detail.

In plan-driven work, change is costly because the plan must be redone.

In agile, change is welcome at any time because work is done in small pieces.

Plan-driven projects produce heavy documentation at every stage.

Agile projects keep documentation light and focus on working software.

The customer is involved mainly at the start and end in plan-driven work, but daily in agile work.

Plan-driven suits fixed, critical systems like flight control software.

Agile suits fast-changing products like mobile apps and websites.

Example: building a bridge follows a plan-driven style, while building a new food delivery app follows an agile style.

In short, plan-driven trusts the plan, agile trusts the team and the feedback.

30 Explain the principles of agility.

IMPORTANT

Agility is the ability of a team to respond quickly and correctly when requirements change.

It is guided by several principles from the Agile Manifesto.

Satisfy the customer through early and continuous delivery of useful software.

Welcome changing requirements, even late in development, because change gives the customer an advantage.

Deliver working software frequently, in short periods of a few weeks.

Business people and developers must work together daily throughout the project.

Build projects around motivated people, give them support, and trust them.

The most efficient communication is face-to-face conversation.

Working software is the main measure of progress.

Simplicity is essential: do the maximum work with the minimum effort.

Teams regularly reflect on how to improve and adjust their behaviour.

Example: a startup that changes its app design after user feedback within one week is showing agility in action.

In short, agility means fast delivery, easy change and close teamwork.

31 State and explain the four values of the Agile Manifesto.

IMPORTANT

The Agile Manifesto states four values that guide all agile work.

First: individuals and interactions over processes and tools.

This means talented people talking to each other matter more than rigid rules and fancy software.

Second: working software over comprehensive documentation.

This means a running product matters more than thick documents nobody reads.

Third: customer collaboration over contract negotiation.

This means working with the customer daily is better than fighting over a fixed contract.

Fourth: responding to change over following a plan.

This means adapting to new needs is better than blindly following an old plan.

Note that the items on the right still have value, but the items on the left are valued more.

Example: a team that quickly rebuilds a feature after customer feedback is following the fourth value.

These four values are the heart of every agile method like Scrum and XP.

TOPIC 9

9

Agile Methods: XP and Scrum

Agile methods are practical ways of working in an agile style.

Extreme Programming (XP): very short cycles, pair programming, continuous testing, simple design.

Scrum: work divided into sprints; roles are Product Owner, Scrum Master and Development Team.

Other methods: Kanban, Crystal, Feature-Driven Development.

QUESTION BANK

32 **Explain Extreme Programming and its practices. [Asked: Model paper]**

REPEATED

Extreme Programming, or XP, is an agile method that takes good practices to an extreme level.

It uses very short development cycles with constant feedback from the customer.

Its key practices are the following.

Pair programming: two developers work on one machine; one writes code while the other reviews it live.

Test-driven development: write the test first, then write the code that passes the test.

Continuous integration: merge and test all code many times a day so errors are found early.

Simple design: keep the design as simple as possible and improve it later.

Refactoring: regularly clean and improve the code without changing what it does.

The customer is always available on site to answer questions quickly.

XP suits small teams building software with changing requirements.

Example: a team of five building a college event app can use pair programming and daily integration to stay fast and correct.

So XP delivers quality software by pushing teamwork, testing and simplicity to the extreme.

33 Compare Extreme Programming with Scrum. [Asked: Model paper]

REPEATED

XP and Scrum are both agile methods, but they focus on different things.

XP focuses on engineering practices: how the code is actually written.

Its core is pair programming, test-driven development, continuous integration and simple design.

Scrum focuses on project management: how the work is organised.

Its core is roles, sprints and meetings like planning, standup, review and retrospective.

XP says little about who manages the project; Scrum says little about how to write code.

XP suits small technical teams that need coding discipline.

Scrum suits teams of any size that need clear planning and tracking.

Because they cover different gaps, many teams use both together.

Example: a team can run Scrum sprints for planning and use XP's pair programming and test-first coding inside each sprint.

In short, Scrum manages the work and XP builds the work, so they fit together well.

34 Explain Scrum with its roles and meetings.

IMPORTANT

Scrum is the most popular agile method; it organises work into fixed-length cycles called sprints.

A sprint is usually 2 to 4 weeks long and ends with working software.

Scrum has three roles.

Product Owner: decides what to build and sets the priority of features.

Scrum Master: removes problems that block the team and protects the Scrum process.

Development Team: the developers who build the software; they organise their own work.

Scrum has four main meetings.

Sprint planning: choose the work for the coming sprint.

Daily standup: a 15-minute meeting on progress and blockers.

Sprint review: demo the finished work to stakeholders.

Retrospective: discuss what went well and what to improve.

Example: a team building a cab booking app can deliver login in sprint one, booking in sprint two, and payment in sprint three.

Scrum keeps the team focused, visible and improving every sprint.

35 Explain the Scrum meetings in detail.

IMPORTANT

Scrum uses four meetings to keep the sprint on track.

Sprint planning happens at the start of the sprint.

The team picks items from the product backlog and decides what can be finished in this sprint.

The result is the sprint backlog, the team's promise for the sprint.

Daily standup is a short 15-minute meeting held every day at the same time.

Each member answers three questions: what did I do yesterday, what will I do today, and what is blocking me.

It is not a status report to a boss; it is the team helping each other.

Sprint review happens at the end of the sprint.

The team demos the working software to the product owner and stakeholders and collects feedback.

Retrospective also happens at the end, after the review.

The team honestly discusses what went well, what went wrong and what to change next sprint.

Example: a retrospective may reveal that testing was rushed, so the next sprint adds more test time.

These four meetings give Scrum its rhythm of plan, work, show and improve.

36 What are product backlog, sprint backlog and increment in Scrum?

IMPORTANT

These three are the main artefacts, or work products, of Scrum.

The product backlog is the full ordered list of everything the product may ever need.

It contains features, fixes and improvements, with the most important items on top.

The Product Owner owns it and keeps it updated.

The sprint backlog is the small list of items chosen for the current sprint.

It is created in sprint planning and belongs to the Development Team.

Nobody changes it during the sprint, so the team can focus.

The increment is the working software produced at the end of the sprint.

It must be potentially shippable, meaning it could be given to the customer.

Each sprint's increment adds to all the previous ones, growing the product step by step.

Example: for a food app, the product backlog has 50 features, the sprint backlog has 5 for this sprint, and the increment is the working login and menu screens.

So the product backlog is the whole dream, the sprint backlog is this sprint's promise, and the increment is the finished result.

TOPIC 10

10

Agile Tool Set

Agile tool set means the tools teams use to work in agile style.

Boards (like Trello or Jira) track tasks moving from to-do to done.

Version control (Git) lets many developers work on the same code.

CI tools build and test code automatically on every change.

Chat and video tools keep remote teams in daily contact.

QUESTION BANK

37 **Explain the agile tool set with examples. [Asked: Model paper]**

REPEATED

The agile tool set is the collection of tools that help a team work in an agile style.

Task boards track work visually: tools like Trello and Jira show tasks moving from to-do to doing to done.

Version control tools like Git let many developers work on the same code safely.

Every change is recorded, so mistakes can be undone and work can be merged.

Continuous integration tools like Jenkins build and test the code automatically on every change.

This finds errors within minutes instead of weeks.

Communication tools like Slack and Zoom keep the team in daily contact.

They support daily standups and quick discussions, especially for remote teams.

Test automation tools run the test suite fast after every change.

Example: a team pushes code to Git, Jenkins auto-builds and auto-tests it, and the board card moves to done automatically.

Together these tools make agile speed and quality possible.

38 What is a Kanban board? How does an agile team use it?

IMPORTANT

A Kanban board is a visual board that shows the status of every task in the project.

It has columns like To Do, Doing and Done.

Each task is written on a card placed in the correct column.

As work progresses, team members move their cards from left to right.

The board can be physical, like a whiteboard with sticky notes, or digital, like Trello or Jira.

Everyone can see the project status at a glance without calling a meeting.

If too many cards pile up in the Doing column, the team knows work is stuck and helps out.

This keeps the work flowing smoothly and stops people from starting too many tasks at once.

Example: a student team building a website puts design, coding and testing cards on the board and moves them daily.

So a Kanban board makes work visible, simple and easy to manage.